Corso di Programmazione Web
Esame - 6 Giugno 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.
- Scrivere nome, cognome, numero di matricola nel foglio dove si svolge l'esame

Sistema per la gestione di prestiti di oggetti tra utenti

Ci è stato richiesto di progettare un'applicazione web che consente agli utenti di **prestare e ricevere in prestito oggetti** (libri, strumenti, videogiochi, ecc.) in modo tracciabile.

Requisiti dell'applicazione

Il sistema deve prevedere:

- Un'API per registrare un nuovo oggetto da prestare (id, nome, descrizione, categoria, disponibilità, id proprietario);
- Un'API per visualizzare la lista degli oggetti disponibili;
- Un'API per richiedere il prestito di un oggetto (id oggetto, id richiedente, data inizio, durata prevista);
- Un'API per vedere la cronologia dei prestiti effettuati o ricevuti su un oggetto.

Si progetti in modo che gli id siano assegnati dal server nel momento in cui si registra un oggetto. Gli id degli utenti (proprietari e richiedenti di prestiti) sono numerici. Il campo disponibilità serve a indicare se l'oggetto è disponibile o no (non la quantità).
Per definire le API, si ipotizzi che il prestito sia automaticamente accettato senza fare verifiche sulla disponibilità.

Domanda 1: Definizione delle API (15 punti)

Per ciascuna API:

- **3 punti totali**: Modelli di dati richieste/risposte con tipi e vincoli (es, massimo, minimo). Spiegare il motivo di eventuali vincoli.
- **2 punti per API**: Descrizione endpoint (URL + metodo HTTP)
- **1 punto per API**: Esempio concreto di richiesta con successo e risposta del server. Si indichi il codice di risposta, ed eventuali JSON di risposta.

Domanda 2: Gestione degli Errori (5 punti)

- **1 punto totale:** Spiegare i codici di errore più rilevanti (400, 404, 500)
- **2 punti per esempio, max 4 punti:** Fornire **due esempi** di richieste che possono causare uno dei precedenti errori nelle API appena progettate.

Domanda 3: Concorrenza (3 punti)

- **3 punti totali:** Il sistema deve impedire che uno stesso oggetto venga prestato contemporaneamente a più utenti. *Come si potrebbe progettare una logica robusta per la gestione della concorrenza su oggetti condivisi?*

MODELLO OGGETTO ⟶ {

"IDOGGETTO": INT : 1, //ASSEGNATO DAL SERVER IN MODO INCREMENTALE, UNIVOCO

"NOME": STR: "TRAPPANO BOSCH", //OBBLIGATORIO, MAX 20 CARATTERI

"DESCRIZIONE": STR: "NUOVO MAI USATO", //OBBLIGATORIO, MAX 500 CARATTERI

"CATEGORIA": STR: "STRUMENTI", //OPZIONALE, VALORI PREDEFINITI (ES. "LIBRO", "STRUMENTO", "GIOCO")

"DISPONIBILITÀ": BOOL: "TRUE", //DEFAULT TRUE, VIENE MODIFICATO QUANDO L'OGGETTO VIENE DATO IN PRESTITO

"IDPROPRIETARIO": INT: 2 // SPECIFICA L'UTENTE PROPRIETARIO

}

MODELLO PRESTITO OGGETTO ⟶ {

"IDRICHIEDENTE": INT: 22 // OBBLIGATORIO, INVIATO NEL BODY

"IDOGGETTO": INT: 1, //RICAVATO DAL PATH /OGGETTO/{IDOGGETTO}

"IDPRESTITO": INT: 1, //ASSEGNATO DAL SERVER, UNIVOCO

"DATAINIZIO": STR: "2026-05-31" //OBBLIGATORIO, FORMATO YYYY-MM-DD

"DURATAPREVISTA": INT: 30, // OBBLIGATORIO, MIN=1, MAX=365

"DATAFINE": STR: "2026-06-30", // CALCOLATO DAL SERVER

"IDPROPRIETARIO": INT: 10, //ASSEGNATO DAL SERVER

}

API 1 - REGISTRARE UN NUOVO OGGETTO DA PRESTARE

POST /OGGETTI →

SI USA POST PERCHÈ L'UTENTE STA CREANDO UNA NUOVA RISORSA NEL SISTEMA. L'ID OGGETTO VIENE ASSEGNATO DAL SERVER

← 201 CREATED

{

"NOME": STR: "TRAPPANO BOSCH",

"DESCRIZIONE": STR: "NUOVO MAI USATO",

"CATEGORIA": STR: "STRUMENTI",

"DISPONIBILITÀ": BOOL: "TRUE",

"IDPROPRIETARIO": INT: 2

}

⟶

{

"IDOGGETTO": 1,

"MESSAGE": "OGGETTO REGISTRATO CON SUCCESSO"

}

API 2 - VISUALIZZARE LA LISTA DEGLI OGGETTI DISPONIBILI

GET /OGGETTI?DISPONIBILITÀ=TRUE →

NESSUN BODY. SI USA GET PERCHE L'API LEGGE DATI GIÁ PRESENTI NEL SISTEMA

← 200 OK

{

"IDOGGETTO": 1,

"NOME": "TRAPPANO BOSCH",

"DESCRIZIONE": "NUOVO MAI USATO",

"CATEGORIA": "STRUMENTI",

"DISPONIBILITÀ": "TRUE",

"IDPROPRIETARIO": 2

}

API 3 - RICHIEDERE IL PRESTITO DI UN OGGETTO

POST /OGGETTI/{IDOGGETTO}/LOANS →

SI USA POST PERCHÈ L'UTENTE STA CREANDO UNA NUOVA RICHIESTA DI PRESTITO

← 201 CREATED

{

"IDRICHIEDENTE": INT: 22, //OBBLIGATORIO

"DATAINIZIO": INT: 2026-05-31, // OBBLIGATORIO, FORMATO YYYY-MM-DD

"DURATAPREVISTA": INT: 7 // OBBLIGATORIO, MIN=1, MAX=365

}

⟶

{

"IDPRESTITO": 15,

"IDOGGETTO": 1,

"IDPROPRIETARIO": 10,

"IDRICHIEDENTE": 22,

"DATAINIZIO": "2026-05-31",

"DURATAPREVISTA": 7,

"DURATAFINE": "2026-06-07",

"STATO": "ACCETTATO"

}

API 4 - VEDERE LA CRONOLOGIA DEI PRESTITI SU UN OGGETTO

GET /oggetti/{idOggetto}/loans →

SI USA GET PERCHÈ L'API LEGGE LA CRONOLOGIA DEI PRESTITI GIÁ SALVATI. NESSUN BODY

← 200 OK

```
{
  "idPrestito": 15,
  "idOggetto": 1,
  "idProprietario": 10,
  "idRichiedente": 22,
  "dataInizio": "2026-05-31",
  "durataPrevista": 7,
  "durataFine": "2026-06-07",
  "stato": "ACCETTATO"
}
```

- 400 BAD REQUEST → INDICA UNA RICHIESTA MALFORMATA O INCOMPLETA DA PARTE DEL CLIENT. IL SERVER NON RIESCE AD INTERPRETARE CORRETTAMENTE LA RICHIESTA. SI USA QUANDO IL BODY È MANCANTE UN PARAMETRO RICHIESTO NON VIENE INVIATO, MANCA UN CAMPO OBBLIGATORIO.

POST /oggetti →

← 400 BAD REQUEST

```
{                                   {
  "nome": "TRAPANO BOSCH",            "error": "Body incompleto: descrizione ed idProprietario sono obbligatori"
  "categoria": "strumenti"    →     }
}
```

- 404 NOT FOUND → INDICA CHE LA RISORSA RICHIESTA NON ESISTE. SI USA QUANDO L'UTENTE RICHIEDE UN OGGETTO CON ID INESISTENTE, L'ENDPOINT NON ESISTE

GET /oggetti/999/loans →

← 404 NOT FOUND

```
{
  "error": oggetto con id 999 non trovato"
}
```

- 422 UNPROCESSABLE ENTITY → INDICA CHE LA RICHIESTA È SINTATTICAMENTE CORRETTA, MA CONTIENE DATI CHE VIOLANO VINCOLI LOGICI O DI VALIDAZIONE. SI USA QUANDO DURATAPREVISTA È MINORE DI 1, DATAINIZIO HA UN FORMATO NON VALIDO

POST /oggetti/1/loans →

← 422 UNPROCESSABLE ENTITY

```
{                                   {
  "idRichiedente": 22,                "error": "dati non validi: data inizio deve avere formato yyyy-mm-dd e durataPrevista deve essere almeno 1"
  "dataInizio": "10 giugno 2026",  →  }
  "durataPrevista": 0
}
```

- 500 INTERNAL SERVER ERROR → INDICA UN ERRORE INTERNO LATO SERVER. LA RICHIESTA PUÓ ANCHE ESSERE CORRETTA, MA IL SERVER NON RIESCE A COMPLETARLA. SI USA QUANDO IL DATABASE NON È RAGGIUNGIBILE, IL SERVER NON RIESCE A SALVARE IL NUOVO OGGETTO

POST /oggetti/1/loans →

← 404 NOT FOUND

```
{                                   {
  "idRichiedente": 22,                "error": "errore interno del server. riprova più tardi"
  "dataInizio": "2026-05-31",  →   }
  "durataPrevista": 7
}
```

CONCORRENZA → PER EVITARE CHE LO STESSO OGGETTO VENGA PRESTATO CONTEMPORANEAMENTE A PIÚ UTENTI, IL SERVER DEVE GESTIRE LA RICHIESTA DI PRESTITO IN MODO ATOMICO. QUANDO ARRIVA UNA RICHIESTA, IL SERVER CONTROLLA CHE L'OGGETTO ESISTA E CHE SIA ANCORA DISPONIBILE. SUBITO DOPO, NELLA STESSA OPERAZIONE, REGISTRA IL PRESTITO ED IMPOSTA DISPONIBILE=FALSE. IN QUESTO MODO DUE RICHIESTE SIMULTANEE NON POSSONO MODIFICARE LO STESSO OGGETTO NELLO STESSO MOMENTO. UNA POSSIBILE SOLUZIONE È USARE UNA TRANSAZIONE SUL DATABASE OPPURE UNA CODA DI RICHIESTE, COSI LE OPERAZIONI SULLO STESSO OGGETTO VENGONO ESEGUITE UNA ALLA VOLTA. SE L'OGGETTO RISULTA GIÁ PRESTATO, IL SERVER RESTITUISCE UN ERRORE, AD ESEMPIO 422